EXERCÍCIO PROPOSTO 1

Liste produtos com preço acima da média geral:

```sql
SELECT p.id_produto, p.nome, p.preco
FROM produto AS p
WHERE p.preco > (
    SELECT AVG(p2.preco)
    FROM produto AS p2
)
ORDER BY p.preco DESC;
```

Result Grid | Filter Rows: | Edit:

id_produto	nome	preco
6	Mochila	150.00
23	Webcam	130.00
29	Livro Algoritmos	125.00
2	Banco de Dados	120.00
18	Teclado	120.00
12	Livro Python	110.00
28	Livro Java	105.00
22	Fone de Ouvido	99.90
11	Livro MySQL	95.00
1	Livro SQL	89.90
24	Carregador	89.90
13	Livro Redes	85.00
30	Suporte Noteb...	80.00
16	Calculadora	79.90
21	Capa Notebook	70.00
17	Mouse	65.00
NULL	NULL	NULL

Localize clientes com ao menos um pedido pago usando IN:

```sql
SELECT c.id_cliente, c.nome
FROM cliente AS c
WHERE c.id_cliente IN (
    SELECT p.id_cliente
    FROM pedido AS p
    WHERE p.status = 'PAGO'
)
ORDER BY c.id_cliente;
```

Result Grid | Filter Rows:

id_cliente	nome
1	Ana Silva
3	Bruno Lima
NULL	NULL

Reescreva a busca usando EXISTS:

```sql
SELECT c.id_cliente, c.nome
FROM cliente AS c
WHERE EXISTS (
    SELECT 1
    FROM pedido AS p
    WHERE p.id_cliente = c.id_cliente
      AND p.status = 'PAGO'
)
ORDER BY c.id_cliente;
```

Result Grid | Filter Rows: | Edit:

id_cliente	nome
1	Ana Silva
3	Bruno Lima
NULL	NULL

Encontre clientes sem pedidos usando NOT EXISTS:

```sql
SELECT c.id_cliente, c.nome
FROM cliente AS c
WHERE NOT EXISTS (
    SELECT 1
    FROM pedido AS p
    WHERE p.id_cliente = c.id_cliente
);
```

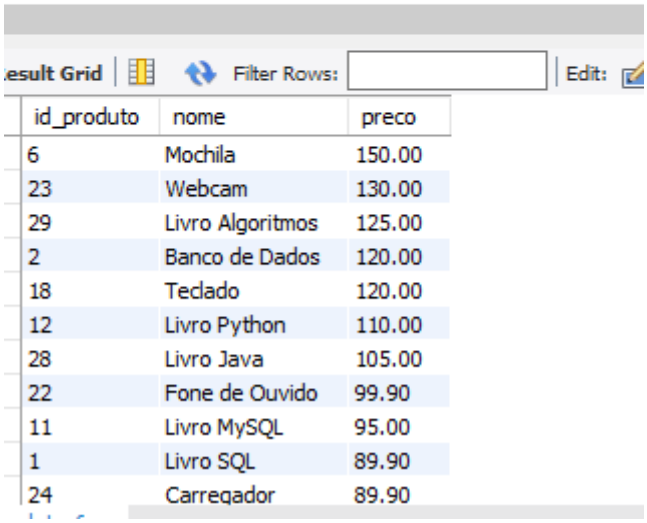
Result Grid | Filter Rows: | Edit:

id_cliente	nome
4	Beatriz Costa
6	Bianca Alves
NULL	NULL

Compare ANY e ALL com os preços da categoria LIVRO:

ANY

```sql
SELECT p.id_produto, p.nome, p.preco
FROM produto AS p
WHERE p.preco > ANY (
    SELECT p2.preco
    FROM produto AS p2
    WHERE p2.categoria = 'Livro'
)
ORDER BY p.preco DESC;
```



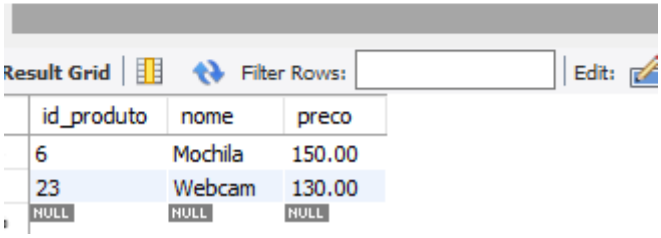
esult Grid | Filter Rows: | Edit:

id_produto	nome	preco
6	Mochila	150.00
23	Webcam	130.00
29	Livro Algoritmos	125.00
2	Banco de Dados	120.00
18	Teclado	120.00
12	Livro Python	110.00
28	Livro Java	105.00
22	Fone de Ouvido	99.90
11	Livro MySQL	95.00
1	Livro SQL	89.90
24	Carregador	89.90

Retorna os produtos cujo preço é maior que pelo menos um dos preços da categoria Livro.

ALL

```sql
SELECT p.id_produto, p.nome, p.preco
FROM produto AS p
WHERE p.preco > ALL (
    SELECT p2.preco
    FROM produto AS p2
    WHERE p2.categoria = 'Livro'
)
ORDER BY p.preco DESC;
```



Result Grid | Filter Rows: | Edit:

id_produto	nome	preco
6	Mochila	150.00
23	Webcam	130.00
NULL	NULL	NULL

Retorna somente os produtos cujo preço é superior ao de todos os livros cadastrados.

EXERCÍCIO PROPOSTO 2

Liste produtos acima da média da própria categoria:

```sql
SELECT p.categoria, p.nome, p.preco
FROM produto AS p
WHERE p.preco > (
    SELECT AVG(p2.preco)
    FROM produto AS p2
    WHERE p2.categoria = p.categoria
)
ORDER BY p.categoria, p.preco DESC;
```

Result Grid | Filter Rows: | Export:

categoria	nome	preco
Acessorio	Mochila	150.00
Acessorio	Suporte Notebook	80.00
Eletronico	Webcam	130.00
Eletronico	Teclado	120.00
Eletronico	Fone de Ouvido	99.90
Eletronico	Carregador	89.90
Livro	Livro Algoritmos	125.00
Livro	Banco de Dados	120.00
Livro	Livro Python	110.00
Livro	Livro Java	105.00
Papelaria	Agenda	25.90
Papelaria	Grampeador	22.00
Papelaria	Caderno	18.50
Papelaria	Pasta	14.90
Papelaria	Bloco de Notas	12.50

Recupere o pedido mais recente de cada cliente:

```sql
SELECT p.id_pedido, c.nome AS cliente, p.criado_em
FROM pedido AS p
JOIN cliente AS c
ON c.id_cliente = p.id_cliente
WHERE p.criado_em = (
    SELECT MAX(p2.criado_em)
    FROM pedido AS p2
    WHERE p2.id_cliente = p.id_cliente
)
ORDER BY p.id_cliente;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Conte

id_pedido	cliente	criado_em
3	Ana Silva	2026-09-15 09:00:00
2	Amanda Souza	2026-09-10 14:20:00
4	Bruno Lima	2026-09-20 16:45:00
5	Carlos Santos	2026-09-25 11:10:00

Reescreva uma correlação usando JOIN com agregação prévia:

```sql
SELECT p.id_pedido, c.nome AS cliente, p.criado_em
FROM pedido AS p
JOIN cliente AS c
ON c.id_cliente = p.id_cliente
JOIN (
    SELECT id_cliente, MAX(criado_em) AS ultima_data
    FROM pedido
    GROUP BY id_cliente
) AS u
ON u.id_cliente = p.id_cliente
AND u.ultima_data = p.criado_em
ORDER BY p.id_cliente;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id_pedido	cliente	criado_em
3	Ana Silva	2026-09-15 09:00:00
2	Amanda Souza	2026-09-10 14:20:00
4	Bruno Lima	2026-09-20 16:45:00
5	Carlos Santos	2026-09-25 11:10:00

Crie uma CTE para calcular faturamento por produto:

```sql
WITH vendas_produto AS (
    SELECT i.id_produto,
           SUM(i.quantidade * i.preco_praticado) AS total_vendido
    FROM item_pedido AS i
    GROUP BY i.id_produto
)
SELECT p.nome AS produto, v.total_vendido
FROM vendas_produto AS v
JOIN produto AS p
ON p.id_produto = v.id_produto
ORDER BY v.total_vendido DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

produto	total_vendido
Livro MySQL	190.00
Livro SQL	179.80
Garrafa	165.00
Mochila	150.00
Calculadora	79.90
Caneta Azul	26.00

Compare clareza, cardinalidade e provável custo das versões:

Clareza: a subconsulta correlacionada facilita a comparação de cada registro. O JOIN organiza os relacionamentos, enquanto a CTE torna o código mais fácil de entender, dividindo a consulta em etapas.

Cardinalidade: a quantidade de linhas retornadas depende das condições utilizadas. Consultas com JOIN podem gerar repetições quando existem várias correspondências.

Provável custo: a subconsulta correlacionada pode exigir cálculos repetidos, tornando a execução mais lenta. O JOIN com agregação prévia pode reduzir esse trabalho. A CTE melhora a organização, mas não garante melhor desempenho.

DESAFIO INTEGRADOR

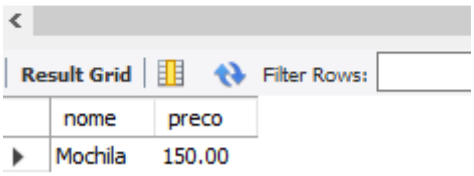
Duas subconsultas escalares

```
46 •  SELECT p.nome, p.preco
47    FROM produto AS p
48    WHERE p.preco > (
49        SELECT AVG(preco)
50        FROM produto
51    )
52    ORDER BY p.preco DESC;
```

Result Grid | Filter Rows:

nome	preco
Mochila	150.00
Webcam	130.00
Livro Algoritmos	125.00
Banco de Dados	120.00
Teclado	120.00
Livro Python	110.00
Livro Java	105.00
Fone de Ouvido	99.90
Livro MySQL	95.00
Livro SQL	89.90
Carregador	89.90
Livro Redes	85.00
Suporte Noteb...	80.00
Calculadora	79.90
Capa Notebook	70.00
Mouse	65.00

```sql
SELECT p.nome, p.preco
FROM produto AS p
WHERE p.preco = (
    SELECT MAX(preco)
    FROM produto
);
```

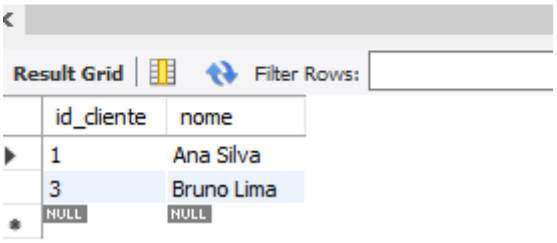


Result Grid | Filter Rows:

nome	preco
Mochila	150.00

Duas subconsultas de múltiplas linhas:

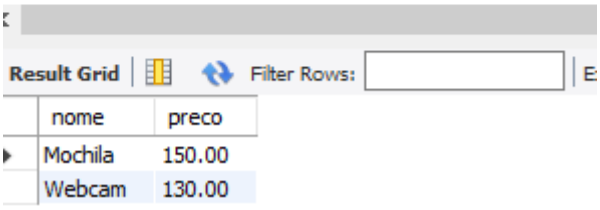
```sql
SELECT c.id_cliente, c.nome
FROM cliente AS c
WHERE c.id_cliente IN (
    SELECT p.id_cliente
    FROM pedido AS p
    WHERE p.status = 'PAGO'
);
```



Result Grid | Filter Rows:

id_cliente	nome
1	Ana Silva
3	Bruno Lima
NULL	NULL

```sql
SELECT p.nome, p.preco
FROM produto AS p
WHERE p.preco > ALL (
    SELECT p2.preco
    FROM produto AS p2
    WHERE p2.categoria = 'Livro'
);
```



Result Grid | Filter Rows:

nome	preco
Mochila	150.00
Webcam	130.00

Uma solução com EXISTS e outra com NOT EXISTS:

```
77 •  SELECT c.id_cliente, c.nome
78    FROM cliente AS c
79    WHERE EXISTS (
80        SELECT 1
81        FROM pedido AS p
82        WHERE p.id_cliente = c.id_cliente
83    );
```

Result Grid | Filter Rows: | Edit:

id_cliente	nome
1	Ana Silva
2	Amanda Souza
3	Bruno Lima
5	Carlos Santos
NULL	NULL

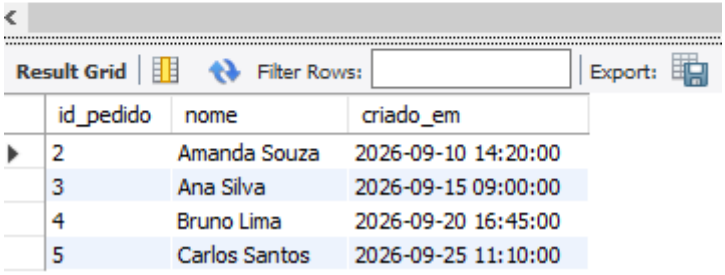
```
85 •  SELECT c.id_cliente, c.nome
86    FROM cliente AS c
87    WHERE NOT EXISTS (
88        SELECT 1
89        FROM pedido AS p
90        WHERE p.id_cliente = c.id_cliente
91    );
```

Result Grid | Filter Rows: | Edit:

id_cliente	nome
4	Beatriz Costa
6	Bianca Alves
NULL	NULL

Uma subconsulta correlacionada e sua reescrita:

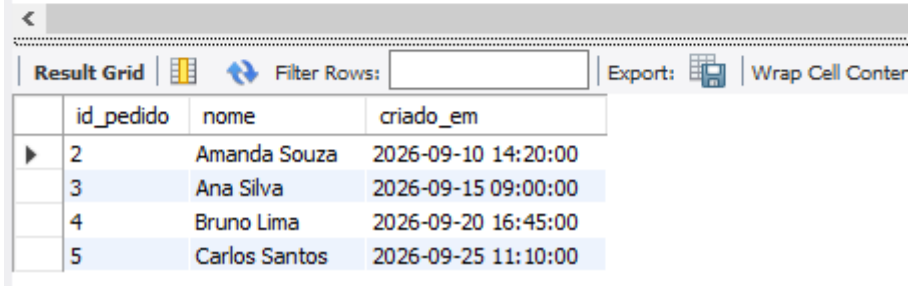
```sql
SELECT p.id_pedido, c.nome, p.criado_em
FROM pedido AS p
JOIN cliente AS c
ON c.id_cliente = p.id_cliente
WHERE p.criado_em = (
    SELECT MAX(p2.criado_em)
    FROM pedido AS p2
    WHERE p2.id_cliente = p.id_cliente
);
```



Result Grid | Filter Rows: | Export:

id_pedido	nome	criado_em
2	Amanda Souza	2026-09-10 14:20:00
3	Ana Silva	2026-09-15 09:00:00
4	Bruno Lima	2026-09-20 16:45:00
5	Carlos Santos	2026-09-25 11:10:00

```sql
SELECT p.id_pedido, c.nome, p.criado_em
FROM pedido AS p
JOIN cliente AS c
ON c.id_cliente = p.id_cliente
JOIN (
    SELECT id_cliente, MAX(criado_em) AS ultima_data
    FROM pedido
    GROUP BY id_cliente
) AS u
ON u.id_cliente = p.id_cliente
AND u.ultima_data = p.criado_em;
```

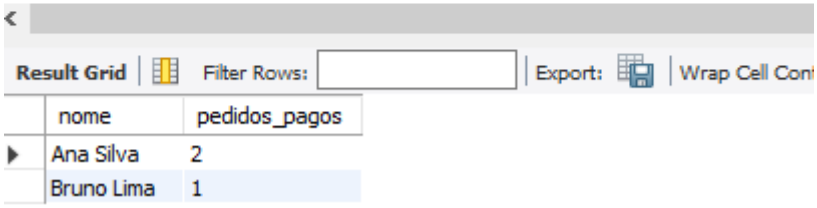


Result Grid | Filter Rows: | Export: | Wrap Cell Conter

id_pedido	nome	criado_em
2	Amanda Souza	2026-09-10 14:20:00
3	Ana Silva	2026-09-15 09:00:00
4	Bruno Lima	2026-09-20 16:45:00
5	Carlos Santos	2026-09-25 11:10:00

Duas CTEs encadeadas para produzir um indicador:

```sql
WITH pedidos_pagos AS (
    SELECT id_cliente, id_pedido
    FROM pedido
    WHERE status = 'PAGO'
),
totais AS (
    SELECT id_cliente,
           COUNT(*) AS quantidade
    FROM pedidos_pagos
    GROUP BY id_cliente
)
SELECT c.nome, t.quantidade AS pedidos_pagos
FROM totais AS t
JOIN cliente AS c
ON c.id_cliente = t.id_cliente
ORDER BY t.quantidade DESC;
```



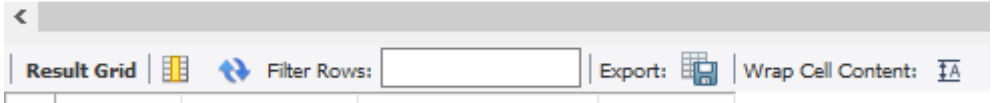
Result Grid | Filter Rows: | Export: | Wrap Cell Cont

nome	pedidos_pagos
Ana Silva	2
Bruno Lima	1

Uma view que simplifique uma consulta recorrente:

```sql
CREATE VIEW vw_pedidos_clientes AS
SELECT p.id_pedido,
       c.nome AS cliente,
       p.criado_em,
       p.status
FROM pedido AS p
JOIN cliente AS c
ON c.id_cliente = p.id_cliente;

SELECT *
FROM vw_pedidos_clientes
ORDER BY id_pedido;
```

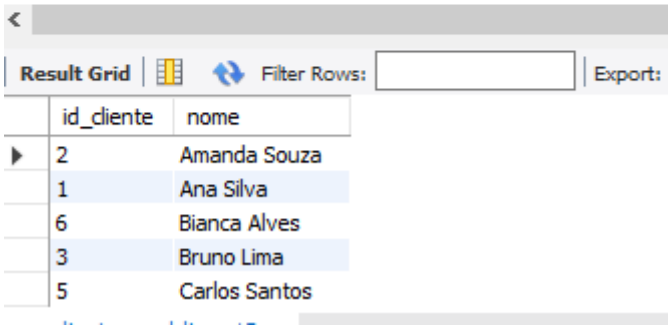


Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id_pedido	cliente	criado_em	status
1	Ana Silva	2026-09-01 10:30:00	PAGO
2	Amanda Souza	2026-09-10 14:20:00	PENDENTE
3	Ana Silva	2026-09-15 09:00:00	PAGO
4	Bruno Lima	2026-09-20 16:45:00	PAGO
5	Carlos Santos	2026-09-25 11:10:00	PENDENTE

Uma view que restrinja colunas expostas

```
146 •   CREATE VIEW vw_clientes_publicos AS
147     SELECT id_cliente, nome
148     FROM cliente
149     WHERE ativo = TRUE;
150
151 •   SELECT *
152     FROM vw_clientes_publicos
153     ORDER BY nome;
```



Result Grid | Filter Rows: | Export:

id_cliente	nome
2	Amanda Souza
1	Ana Silva
6	Bianca Alves
3	Bruno Lima
5	Carlos Santos

Justificativa dos privilégios: o acesso pode ser limitado ao privilégio SELECT na view, evitando a exposição direta de dados pessoais. Para que essa restrição seja efetiva, os usuários não devem possuir acesso direto às tabelas de origem quando a política exigir.